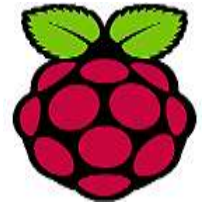


TP1 : OS embarqué pour IoT

Préparation de l'environnement :



1. Installation de l'OS sur la carte SD
2. Connexion à Rpi via ssh :
 - *nmap -version*
 - si NMAP n'est pas installé : *winget install Nmap.Nmap*
ou bien : *choco install nmap -y*
 - *Fermer le terminal et redémarrer, puis, nmap -version*
 - *Trouver votre réseau local : ipconfig*
et chercher : IPv4 Address : 192.168.1.34
Subnet Mask : 255.255.255.0 → réseau = 192.168.1.0/24
 - Scanner le réseau avec nmap : *nmap -sn 192.168.1.0/24*
Ou bien nmap -p 22 192.168.1.0/24 (scanne seulement les machines avec ssh connexion)
 - *ssh pi@Rpi_IP* (entrer user name & Pwd)
3. Mise à jour du système : *sudo apt update | sudo apt upgrade -y*
4. Installation des outils nécessaires :
sudo apt install python3 python3-pip -y |
5. Création de dossier et fichiers
*mkdir -p TP1_OS_IoT && cd TP1_OS_IoT && touch demo_multitache.py
demo_Bare_metal.py demo_sans_priorite.py demo_priorite_OS.py
demo_sans_gestion.py demo_gestion_OS.py*

ls

DÉMO A : Multitâche

Problème : “Comment faire plusieurs choses en même temps ?”

Objectif : Montrer le temps d'exécution de deux instructions sans at avec OS embarqué

nano demo bare metal.py

Fichier 1 : demo_bare_metal.py

```
import time
def lire_capteur():
    print("[CAPTEUR] Début de lecture... ")
    time.sleep(2)  # Simulation de lecture capteur / 2s
    print("[CAPTEUR] ✓ Fini !")

def envoyer_wifi():
    print("[WIFI] Début envoi... ")
    time.sleep(3)  # Simulation d'envoi réseau / 3s
    print("[WIFI] ✓ Fini !")

# VERSION SÉQUENTIELLE (bare-metal)

debut = time.time()

lire_capteur()  # BLOQUE pendant 2s
envoyer_wifi()  # BLOQUE pendant 3s

duree = time.time() - debut
print(f"\n Temps total: {duree:.1f} secondes\n")
```

nano demo multitache.py

Fichier 2 : demo_multitache.py

```
import time
import threading

def lire_capteur():
    print("[CAPTEUR] Début lecture... ")
    time.sleep(2) # 2s
    print("[CAPTEUR] ✓ Fini !")

def envoyer_wifi():
    print("[WIFI] Début envoi...")
    time.sleep(3) # 3s

    print("[WIFI] ✓ Fini !")

# VERSION PARALLÈLE (avec OS)
debut = time.time()

# Créer 2 threads (= 2 tâches indépendantes)
thread_capteur = threading.Thread(target=lire_capteur)
thread_wifi = threading.Thread(target=envoyer_wifi)

# Démarrer Les 2 threads EN MÊME TEMPS
thread_capteur.start()
thread_wifi.start()

# Attendre qu'ils finissent tous les deux
thread_capteur.join()
thread_wifi.join()

duree = time.time() - debut
print(f"\n Temps total: {duree:.1f} secondes\n")
```

DÉMO B : Gestion des priorités

Problème : “Qui passe en premier quand tout arrive en même temps ?”

Objectif : Montrer l’interruption immédiate pour prioriser une tâche urgente

nano demo_sans_priorite.py

Fichier 3 : demo_sans_priorite.py

```
import time

def tache_lente():
    print("[WIFI] Envoi d'une grosse vidéo... 5s")
    for i in range(5):
        time.sleep(1)
        print(f"    ... progression {i+1}/5")
    print("[WIFI] ✓ Terminé")

def alarme():
    print("ALARME INCENDIE ! Action immédiate requise !")

# VERSION SANS PRIORITÉ / exécution séquentielle
print("Début envoi vidéo...\n")
tache_lente() # Prend 5 secondes
print("\n Alarme bloquée :")
alarme()
print()
```

Fichier 4 : demo_priorite_OS.py

```
import time
import threading

# Variable globale pour simuler une alarme
alarme_activee = False

def tache_lente():
    print("[WIFI] Envoi d'une grosse vidéo... 5s")
    for i in range(5):
        # Vérifie si alarme pendant l'envoi
        if alarme_activee:
            print("WiFi mis en pause (priorité basse)")
            return
        time.sleep(1)
        print(f"    ... progression {i+1}/5")
    print("[WIFI] ✓ Terminé")

def alarme():
    global alarme_activee
    time.sleep(2) # Alarme arrive après 2 secondes
    alarme_activee = True
    print("\nALARME INCENDIE ! (priorité haute)")
    print("    → Interruption immédiate de WiFi\n")

# VERSION AVEC PRIORITÉ

thread_wifi = threading.Thread(target=tache_lente)
thread_alarme = threading.Thread(target=alarme)

thread_wifi.start()
thread_alarme.start()

thread_wifi.join()
thread_alarme.join()
print()
```

DÉMO C : Gestion de l'énergie

Problème : “Comment économiser la batterie sans perdre en réactivité ?”

Objectif : Montrer le tournement à vide **VS** la veille et la réveille automatique du CPU sur un événement

- Un capteur IoT doit attendre 1 minute avant d'envoyer des données.
Sans gestion d'énergie intelligente

nano demo_sans_gestion.py

Fichier 5 : demo_sans_gestion.py

```
import time

print("Le CPU tourne à vide en attendant...\n")

compteur = 0
debut = time.time()

# Attente active (busy waiting)
while time.time() - debut < 3:
    compteur += 1 # CPU calcule inutilement
    pass # Boucle vide = gaspillage

print(f"CPU a fait {compteur:}, boucles inutiles")
print("    Batterie drainée pour rien !\n")
```

nano demo_gestion_OS.py

Fichier 6 : demo_gestion_OS.py

```
import time
import signal
import sys

def reveil(sig, frame):
    print("Événement détecté ! CPU se réveille")
    print("    → Traite l'événement rapidement")
    print("    → Retourne en veille\n")
    sys.exit(0)

# Configure Le réveil sur Ctrl+C
signal.signal(signal.SIGINT, reveil)

print("CPU en veille profonde (appuyez Ctrl+C pour simuler un événement)")
print("    Économie de 70-80 % de batterie")

try:
    while True:
        time.sleep(100) # OS dort automatiquement
except KeyboardInterrupt:
    pass
```

Synthèse ?!